
What Does a Benign-Trained Anomaly Detector Actually Detect?

Suramya Angdembay
University of Southern Mississippi

suramya.angdembay@usm.edu

Haiyan Tian
University of Southern Mississippi

haiyan.tian@usm.edu

Abstract

High ranking performance does not reveal what an anomaly detector uses to distinguish malicious activity from normal activity. We audit a language model adapted only to benign employee logs by decomposing its internal changes into sparse features, intervening on selected features, and measuring where they activate. On CERT r6.2, the selected features activate almost entirely at personality and organizational fields; on r4.2, four of five concentrate on behavioral session fields. Comparing malicious days only with benign users unseen during adaptation reduces day-level ranking quality from 0.95 to 0.55 and from 0.96 to 0.67, respectively. The attribution pattern survives new dictionaries and smaller-model replications, while some intervention effects do not. A real-user study with personality questionnaires supplies a behavioral counterpart; a real authentication log shows a related generalization gap that is sensitive to computer identifiers. Removing profile content from both adaptation and scoring improves user-level ranking from 0.53 to 0.90 in the audited Qwen3-8B run, with a smaller change in Qwen2.5-3B. These comparisons use one adapter seed per condition and only four malicious CERT r6.2 users, so they establish a result for those runs rather than a general capacity effect. The contribution is an audit that separates ranking quality, causal relevance, and the input locations of selected features, while making the limits of each measurement explicit.

1 The problem

Organizations record what employees do on their computers: logons, file transfers, web activity, e-mail volume, and so on. An *insider threat* is an employee who misuses legitimate access, for example by stealing data. Because such events are rare, a common approach is *anomaly detection*: build a statistical model of normal activity, and flag days that the model finds unusual.

Formally, a detector is a function s that assigns a real-valued *anomaly score* $s(x)$ to each observation x , where in our setting x is a record of one user’s activity on one day. Higher scores are meant to indicate more suspicious activity. Detectors are usually evaluated by how well their scores *rank* malicious days above benign ones. The standard summary of ranking quality is the following probability, estimated over a labeled test set:

$$\text{AUC} = \Pr(s(X_{\text{mal}}) > s(X_{\text{ben}})) + \frac{1}{2} \Pr(s(X_{\text{mal}}) = s(X_{\text{ben}})), \quad (1)$$

where X_{mal} is a randomly chosen malicious day and X_{ben} a randomly chosen benign day. (This quantity is commonly called the ROC-AUC, for “area under the receiver operating characteristic curve”; the probability formulation is equivalent and is the only form we need; ties receive half credit.) An AUC of 1 means perfect ranking and 0.5 means this ranking measure equals its chance reference.

The question that motivates our work is simple to state. A high AUC says that the score *separates* the two classes. It does not say *on the basis of what information* the separation is achieved. Two detectors with identical AUC could work very differently: one might respond to genuinely unusual behavior, while

the other might respond to incidental properties of *who* the malicious users happen to be, such as their role, department, or personality profile. Such a shortcut can fail when the relationship between identity and malicious activity changes, including when the detector encounters new users. This paper develops a method for determining which kind of detector one actually has, applies it to a modern detector built from a large language model, and then subjects the answer to a systematic robustness program.

This question connects anomaly detection (Chandola et al., 2009) with shortcut learning (Geirhos et al., 2020; Lapuschkin et al., 2019): good predictions can rely on unintended cues. Security evaluation also requires careful control of data splits and experimental assumptions (Arp et al., 2022; Landauer et al., 2024). Our contribution is the combined audit of adaptation changes, feature interventions, input locations, and user-disjoint evaluation; we do not introduce a new sparse-coding method or claim a detection leaderboard advance. Appendix G places this combination alongside earlier log detectors and interpretability methods.

2 The data

We use two versions, r4.2 and r6.2, of the CERT insider-threat corpus, a synthetic enterprise dataset produced by Carnegie Mellon University (Lindauer, 2020; Glasser & Lindauer, 2013). Raw activity is aggregated into session features following the approach of Le et al. (2020), then grouped into user-days. Our evaluation covers only days represented in those extracted sessions. This distinction changes the population: r6.2 has 70 positive user-days from 4 malicious users in our matched domain, compared with 99 positive answer-key rows from 5 users; r4.2 has 1,309 days from 60 users, compared with 1,883 rows from 70 users. Incidents outside session coverage are not evaluated. We later examine two real datasets, TWOS and LANL (Sections 7 and 8).

Each user-day is converted into a short structured text with three kinds of lines:

- a **DAY** line holding the week and organizational fields for that user (project, role, business unit, department, team, and an information-technology administrator flag);
- a **PSY** line holding the user’s five simulated personality scores. These are the “Big Five” personality traits used in psychology: openness (O), conscientiousness (C), extraversion (E), agreeableness (A), and neuroticism (N). In CERT these numbers are generated by the simulator and are constant for each user;
- one **SES** line per work session, holding numeric behavioral counts for that session: duration, number of logons, file and removable-drive activity, e-mail volume, web-browsing category counts, and similar quantities.

A user-day text is therefore a small table serialized as text. The first two lines contain the profile fields, most of which are constant across a user’s days, while the session lines describe daily activity. The text also contains a session-count line, labeled **SESCOUNT**. We use *profile* for the **DAY/PSY** classes and *behavioral* for **SES**; these are locations in the input, not assumptions about the exact computation represented there. This partition of the input is the reference frame for the entire paper.

Users are split as follows. Users with any malicious day are reserved entirely for evaluation. The remaining benign users are divided into a training set (roughly ninety percent) and a validation set (roughly ten percent) whose days never enter training. The validation users will play an important role: they provide a comparison group absent from adaptation. Some scored evaluation users also have no positive day within the matched domain; Appendix A specifies the populations used in each test.

3 The detector

3.1 Language models as probability distributions

A *language model* is a parametric probability distribution over sequences of symbols called tokens. Writing a sequence as $x = (x_1, \dots, x_n)$, the model has the product form

$$p_\theta(x) = \prod_{t=1}^n p_\theta(x_t \mid x_1, \dots, x_{t-1}), \quad (2)$$

where each conditional distribution is computed by a neural network with parameters θ . Modern large language models are such networks with billions of parameters, trained on enormous general text corpora. We use Qwen3-8B, an eight-billion-parameter model (Qwen Team, 2025). The network processes a sequence through L successive layers; at layer ℓ and position t it maintains an internal state vector $h_t^{(\ell)} \in \mathbb{R}^d$ with $d = 4096$. These internal vectors matter later: they are the objects we analyze.

3.2 Adapting the model to normal behavior only

We specialize the general-purpose model to our corpus by *fine-tuning*: continuing its training on the serialized user-day texts, using only days from benign training users. The objective is maximum likelihood, that is, minimizing the negative log-likelihood of the benign corpus $\mathcal{D}$:

$$\mathcal{L}(\theta) = - \sum_{x \in \mathcal{D}} \sum_t \log p_\theta(x_t \mid x_{<t}). \quad (3)$$

No malicious example is ever used in training. This is the classical one-class setup: model the normal population, then measure deviation from it.

For efficiency the fine-tuning does not modify all eight billion parameters. Each adapted attention or feed-forward projection matrix W is kept fixed and receives a trainable low-rank correction,

$$W' = W + \frac{\alpha}{r} B A, \quad B \in \mathbb{R}^{d_{\text{out}} \times r}, \quad A \in \mathbb{R}^{r \times d_{\text{in}}}, \quad (4)$$

with rank $r = 16$, so only the small matrices A and B are learned. This technique is called LoRA (low-rank adaptation) (Hu et al., 2022); we use a memory-saving variant called QLoRA in which the fixed weights are stored at reduced numerical precision (Detrmers et al., 2023). The pair (fixed base model, learned low-rank correction) is called the *adapted model*. The important point is that the learned weight updates are carried by removable low-rank adapters, which makes before-versus-after comparisons clean.

3.3 The anomaly score and two ways to evaluate it

The score of a day x is its average surprisal under the adapted model:

$$s(x) = - \frac{1}{n-1} \sum_{t=2}^n \log p_{\theta_{\text{ada}}}(x_t \mid x_{<t}). \quad (5)$$

Here the first input token supplies context; the remaining tokens are scored as next-token targets, with padding excluded. Days that the model of normal behavior finds improbable receive high scores.

We compare two evaluation populations. Their difference is part of the finding, because they ask different questions about generalization.

1. *The original fold-aligned comparison.* Malicious days are compared against benign days drawn from the general population. Because the model was trained on about ninety percent of benign users, most of those benign comparison days belong to users the model saw during training. Under this protocol the adapted model looks excellent: AUC about 0.953 on r6.2 and 0.964 on r4.2, higher

than four standard anomaly-detection baselines evaluated on the same benchmarks (an isolation forest, a deep one-class method called Deep SVDD, and two recurrent autoencoders), whose AUC values range from about 0.63 to 0.77. Here a fold contains one malicious user and 800 benign users. The language model is held fixed across folds, whereas the baselines retrain with those test users excluded: the training exposure is unequal.

2. *The user-disjoint protocol.* Malicious days are compared only against benign days of the validation users, whom the model never saw. Malicious users are unseen by construction, so under this protocol neither side contributes user-days to adaptation.

The second comparison removes the imbalance in training exposure. It does not make unseen profiles identical across classes or isolate familiarity from every other population difference. We report the gap in Section 5.2 and test profile sensitivity separately.

AUC also does not specify how many alerts are correct. We therefore report *average precision* (AP), which averages precision as the score threshold admits more positives. The result plots label this quantity PR-AUC, following the evaluation code. Its random-ranking reference is the positive fraction of the evaluated population; it is not precision at a single operational threshold. Appendix B gives the full baseline comparison.

4 Opening the model

We now describe the four analysis steps, in the order they are applied. Together they connect the model’s internal changes to measurable score effects and input locations.

Figure 1 summarizes the sequence.

4.1 Step 1: isolate what fine-tuning changed

Because the base model is kept frozen, we can run the same day x through both the base and adapted models and subtract their internal states. Define, at a chosen layer ℓ and each token position t ,

$$\delta_t = h_t^{(\ell),\text{ada}} - h_t^{(\ell),\text{base}} \in \mathbb{R}^d. \quad (6)$$

The vector δ_t isolates the representational change induced by the adaptation at the audited layer and position. It is the natural object to study if one wants to know what the adaptation, as opposed to the general-purpose model, contributes at that point in the computation.

4.2 Step 2: approximate the changes with a sparse dictionary

The vectors δ_t live in $\mathbb{R}^{4096}$ and are not individually interpretable. We therefore approximate them by sparse nonnegative combinations of learned dictionary elements. After standardizing each coordinate (subtracting its mean μ and dividing by its standard deviation σ), we seek a dictionary matrix $D \in \mathbb{R}^{d \times m}$ (with m columns, called *features*; $m = 4d$ for the headline r6.2 audit and $m = 2d$ for the native r4.2 audit) and, for each standardized vector $\tilde{\delta}_t$, a coefficient vector $z_t \in \mathbb{R}_{\geq 0}^m$ with at most k nonzero entries ($k = 8$ and $k = 4$ respectively), such that

$$\tilde{\delta}_t \approx Dz_t, \quad (7)$$

with D and the coefficients chosen to minimize the mean squared reconstruction error over a large sample of token positions. This is a sparse coding problem; the particular parametrization we use (a one-layer encoder network followed by a keep-the-largest- k rule and a linear decoder) is called a *sparse autoencoder* in the machine-learning literature (Cunningham et al., 2023; Gao et al., 2024). The outcome is that each change vector is expressed as a short combination of reusable directions, and each direction can be studied on its own. One technical point deserves notice: because the standardization constants differ between datasets, any geometric comparison of features across datasets must first map the dictionary columns back to the original coordinates, that is, compare the directions $\sigma \odot D_{:,f}$ rather than the raw columns. All cross-dataset comparisons in this paper do so.

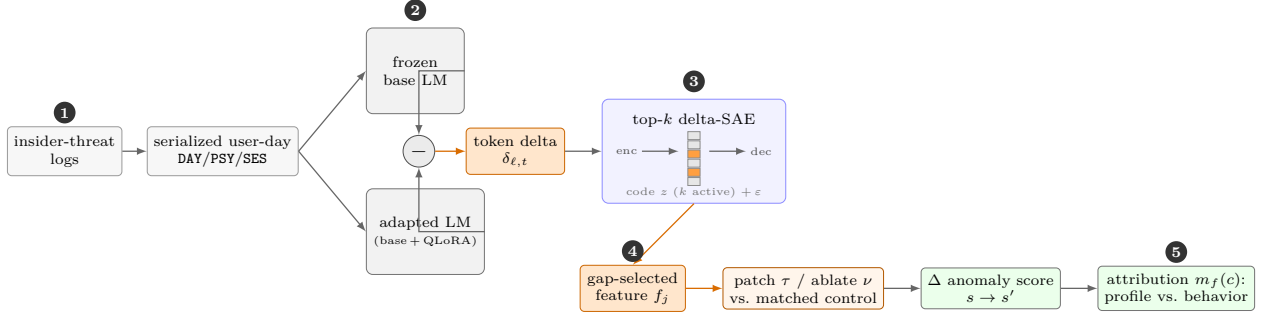


Figure 1: The delta-SAE audit, drawn as architecture. **(1)** A user-day is serialized into structured DAY/PSY/SES text. **(2)** The same input runs through the frozen base model and its benign-only QLoRA-adapted copy; at layer ℓ we take the per-token residual-stream difference $\delta_{\ell,t} = h_{\ell,t}^{\text{ada}} - h_{\ell,t}^{\text{base}}$ (the adapter is left abstract—the delta is the object of study). **(3)** δ feeds a top- k sparse autoencoder (encoder $\rightarrow$ sparse code z with only k units active, shown as the lit cells $\rightarrow$ decoder, plus reconstruction error), whose decoder columns form the feature dictionary. **(4)** A gap-selected feature f_j is causally patched (τ) or ablated (ν) against a matched active control, and the effect on the adapted anomaly score $s \rightarrow s'$ is measured. **(5)** Token attribution $m_f(c)$ labels the feature’s content as profile- or behavior-bound. A mechanistic claim requires both causal relevance (Step 4) *and* content (Step 5); formal definitions are in the main paper.

4.3 Step 3: select the score-relevant features and test them causally

We use a simple screening rule to propose candidate features: the difference in mean activation between malicious and benign token rows. A small mean difference does not rule out a more complex contribution. The ranking statistic is

$$g_f = \mathbb{E}[z_{t,f} \mid \text{malicious day}] - \mathbb{E}[z_{t,f} \mid \text{benign day}], \quad (8)$$

and we take the five features with the largest g_f as the candidate set S . As a comparison group we also select a matched *control* set C of five features with nontrivial activation frequency and a small absolute gap $|g_f|$. This active control avoids comparison with features that never fire; closer matching of edit magnitude is a separate check. Selection by a statistic can be flattered by chance, so everything that follows treats S as a hypothesis to be tested rather than a conclusion.

The test is interventional. The idea is to edit the coefficients of the selected features inside the model, let the edited internal state flow through the remaining layers, and observe whether the anomaly score responds. We use two complementary edits.

Repair (a sufficiency-style test). For a malicious day, we move the selected coefficients toward the values they take on a comparable benign day (a “donor” from the same organizational context, never the same user), with an interpolation strength $\alpha \in \{0.25, 0.5, 0.75, 1\}$, and record the best achievable repair over the available donors and strengths. Write Δ_{ben}^S for the best *signed score change*, $s_{\text{edited}} - s_{\text{original}}$, so a reduction is negative, and Δ_{anom}^S for the analogous quantity when donors are other malicious days. If the selected features carry anomaly-specific repairable structure, moving them toward benign values should reduce the score more than moving them toward other anomalous values, and it should do so more for S than for the control set C . The reported estimand is the doubly differenced quantity

$$\tau = (\overline{\Delta_{\text{anom}}^S} - \overline{\Delta_{\text{ben}}^S}) - (\overline{\Delta_{\text{anom}}^C} - \overline{\Delta_{\text{ben}}^C}). \quad (9)$$

Each bar averages the same receivers with all four required outcomes. This paired construction avoids subtracting means over different subsets when donors are missing. It controls some generic effects of editing, but unequal edit sizes and nonlinear responses need not cancel. The operation also replaces the original delta with its sparse reconstruction at every token in both arms. That shared replacement is not a guarantee of exact cancellation; the equations and sensitivity checks are in Appendix C.

Ablation (a necessity-style test). We scale the selected coefficients toward zero, on malicious days and on matched benign days, and ask whether removal disrupts the score more on the malicious side and more for S than for C . The corresponding double difference is denoted ν ; its signed definition is in Appendix C. Repair establishes the usefulness of an edit given the rest of the network, and ablation establishes dependence under that edit. Neither proves that a feature set is sufficient by itself or strictly necessary in a network with redundant pathways (Heimersheim & Nanda, 2024; Zhang & Nanda, 2023).

Two safeguards are essential and were arrived at the hard way. First, donors from the same user are excluded: allowing them lets the procedure “repair” a malicious day by borrowing the same person’s own benign signature, which inflates effects for reasons unrelated to the question. Second, uncertainty is computed by resampling *users*, not days: multiple days of the same user are statistically dependent, so the primary intervention intervals resample malicious users with replacement (10,000 draws) and recompute the paired effect. The masking analysis resamples users within both label groups. Intervals are conditional on the trained models and selected configurations. On r6.2, which has only four malicious users, such intervals are reported as descriptive.

One further check matters because the selected features are, by construction, strongly activated: their edits are physically larger than edits of the control features. We therefore repeated the headline comparisons with *dose-matched* controls, chosen to match the selected features on activation frequency, activation magnitude, and dictionary-direction norm, which closes the edit-size gap to a factor of 1.3 to 1.8. The contrasts survive: on r4.2 the repair contrast remains clearly positive (+0.0017 with interval [0.0011, 0.0023]) and on r6.2 the ablation contrast remains clearly positive (+0.061 with interval [0.007, 0.084]); the remaining two contrasts stay positive but with intervals containing zero. Matching reduces rather than eliminates the difference in edit strength. Dividing effects by edit norm gives a less favorable r6.2 repair result; that sensitivity is retained in Appendix C. Figure 2 summarizes the intervention measurements.

4.4 Step 4: read off what the features respond to

Finally we ask *where* the selected features activate within the day text. Each token position belongs to DAY, PSY, SESCOUNT, or SES. The fractions below are computed over positive-example token rows. For a feature f , define its *attribution mass* on class c as the fraction of its total activation that occurs at positions of that class:

$$m_f(c) = \frac{\sum_{(x,t) : \text{class}(t)=c} z_{t,f}(x)}{\sum_{(x,t)} z_{t,f}(x)}. \quad (10)$$

Profile-position mass is $m_f(\text{DAY}) + m_f(\text{PSY})$; behavioral-position mass is $m_f(\text{SES})$. We also compare each fraction with the fraction of tokens at that location, so that a long line class does not receive credit merely for occupying more of the input.

These measurements answer different questions. Interventions test whether editing a selected set changes the score. Attribution locates where that set activates. A hidden state at a session token can still depend on an earlier profile, so location does not fully identify a feature’s meaning. Nor are the five selected features a complete account of the model: 100 percent profile mass within them is not 100 percent of the detector’s decision. We use input-level swaps, masking, and within-user evaluation to help interpret the combined evidence.

5 Findings on the two CERT benchmarks

5.1 Similar ranking, different selected features

The two CERT versions produce nearly identical headline AUC values, yet the analysis finds qualitatively different selected feature sets.

On r6.2, the five selected features place between 99.8 and 100 percent of their attribution mass on the personality and organizational lines, with activation on the personality line enriched about thirteen times over its share of tokens. The intervention contrasts are positive (repair $\tau = 0.0068$ with descriptive interval

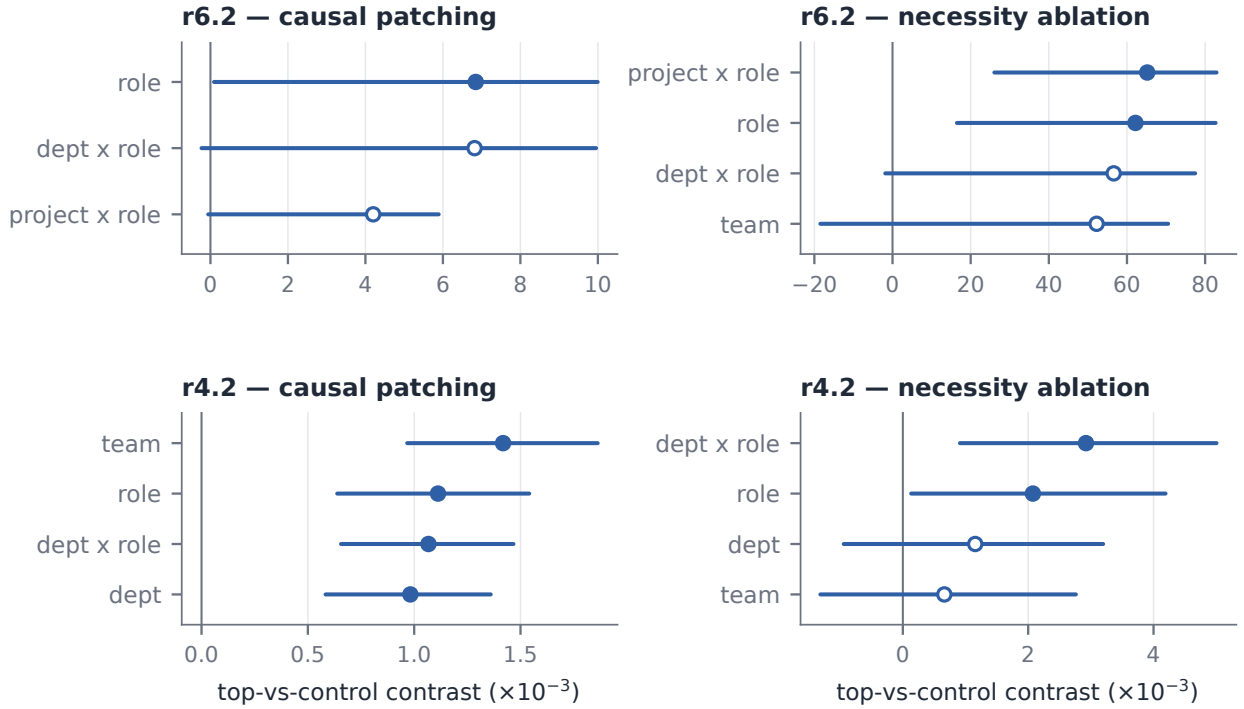


Figure 2: Effect of editing the selected features, compared with editing matched control features, on the anomaly score (one panel per benchmark and per type of edit; units are thousandths of the score). Each point is the average paired contrast for one organizational matching context, and each horizontal bar is a 95 percent confidence interval computed with the malicious users as the resampling unit. Positive values favor the selected set in the specified donor-type or receiver-class contrast, relative to the active control. Open circles mark intervals that include zero.

[0.0001, 0.0100]; ablation $\nu = 0.065$ with interval [0.026, 0.083]). The five gap-ranked features concentrate on profile positions; their intervention contrast is evidence of score relevance, not a ranking of the five largest individual causal effects.

On r4.2, four of the five selected features concentrate on the behavioral session lines (the fifth responds to the organizational header). The contrasts are again positive (repair $\tau = 0.0014$ with interval [0.0010, 0.0019]; ablation $\nu = 0.0029$ with interval [0.0009, 0.0050]). The selected set therefore concentrates on the session portion of the input. Figure 3 shows the contrast, together with the real-data result of Section 7.

5.2 The consequence for the headline numbers

Why would reading a user’s static profile produce a high AUC at all? The adapted model was trained on roughly ninety percent of the benign users. For those users, the model has been repeatedly exposed to the constant profile lines, which may make those profiles easier to predict. The malicious users were never seen in training, so unfamiliar profile text could raise their scores even without unusual activity. This offers a familiarity hypothesis for some of the score gap, which the following comparisons test.

The user-disjoint protocol of Section 3.3 removes this asymmetry, and the effect is large. The AUC falls from 0.954 to 0.545 on r6.2, which is barely better than chance, and from 0.959 to 0.668 on r4.2. These values fall below the baselines’ original fold-aligned AUCs; because the populations differ, this is evidence against a detector superiority claim rather than a new controlled leaderboard. Two further measurements complete the picture. First, ranking each malicious user’s own days (their malicious days against their own benign days, identity held fixed) still succeeds moderately on both benchmarks (average within-user AUC 0.67 on r4.2 and 0.73 on r6.2), so the score contains variation within identities that a static profile-only score could not supply.

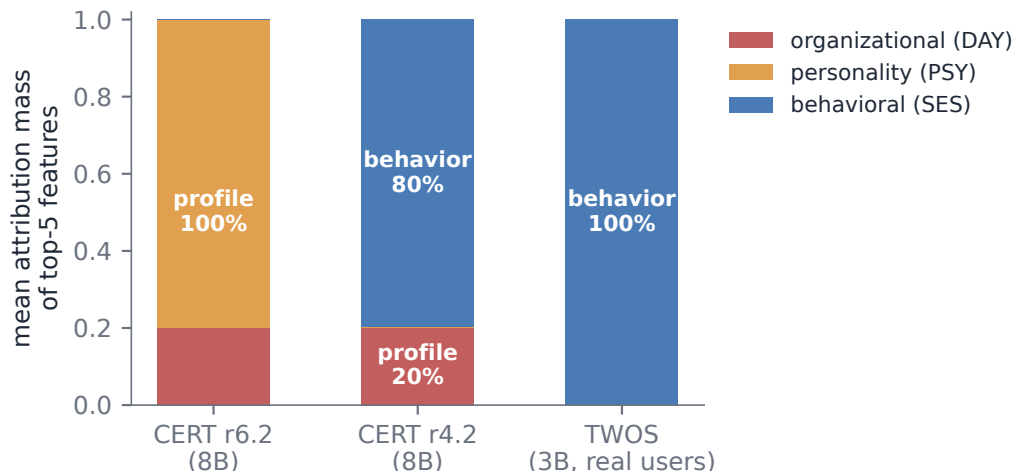


Figure 3: Where the selected features place their activation, averaged over each detector’s five selected features. Each bar decomposes the activation mass into the organizational lines, the personality lines, and the behavioral session lines of the input text. The selected features concentrate on profile positions in r6.2 and on session positions in r4.2 and TWOS. These are averages over features, not shares of the whole model’s decision.

The within-user intervals and counts are given in Appendix B. Second, deleting the personality lines from the inputs at scoring time *improves* the r4.2 detector’s performance on unseen users from 0.67 to about 0.80 (an increase of 0.127 with interval $[0.061, 0.193]$): including the profile text lowered discrimination in that fixed-adaptor comparison. On r6.2, the masking contrasts have intervals spanning zero. This experiment does not show that all profile information is uninformative or that behavioral information is absent; the within-user result already rules out the latter extreme. The masking population and all three conditions are specified in Appendix B. Figure 4 shows the baseline comparison and the collapse.

5.3 Can the same features be reused across benchmarks?

Directly applying the r6.2 dictionary and feature sets to r4.2 fails: the repair contrasts are negative across all audited transfer configurations, with best values around -0.001 . Searching within r4.2 instead finds the positive layer-26 result above. The audit can therefore be reused, but the same sparse features did not transfer in these runs. Seed and matched-layer comparisons below test whether this failure is simply an accident of dictionary initialization. A local session autoencoder supplies another comparator, but its score has different units; raw effect sizes across those models are not a performance ratio (Appendix D.1).

6 Is the interpretation robust?

The checks below separate a stable finding from a result that depends on how it was measured. The consistent result is the contrast in selected activation locations. Repair and ablation do not replicate equally well. Appendix D gives the supporting numbers.

Does selection work on new users? On r4.2, features selected with thirty malicious users were evaluated on the other thirty. Four of five features reappear. Held-out repair effects are positive but about half as large as the exploratory effects; only the department context has a user-bootstrap interval excluding zero. Held-out ablation is positive in some contexts, while team is a null. On r6.2, leaving one malicious user out of selection shows that repair concentrates on the user who contributes 46 of the 70 positive days. Held-out ablation of the original profile features does not transfer to that dominant user. The dictionary and configuration were chosen using the full population, so these checks establish independence of feature selection and testing conditional on those choices, not a wholly independent rediscovery.

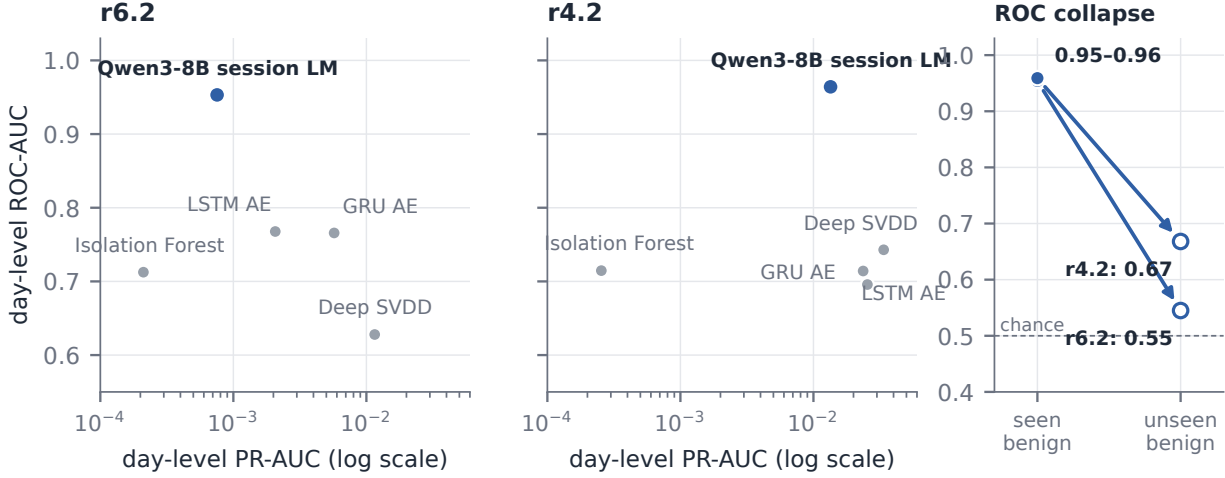


Figure 4: Score-level results. Left and center panels: ranking quality (vertical axis) and average precision (horizontal axis, logarithmic scale) for the language-model detector (blue) and four standard anomaly-detection methods on the two benchmarks; the language model attains the highest ranking quality under the original fold-aligned protocol. The baseline abbreviations are expanded in Appendix B. Right panel: the same language-model ranking quality when malicious users are compared only against benign users never seen during training. The apparent advantage largely disappears, and the dashed line marks chance level.

Is the dictionary an accident of initialization? Three dictionary seeds per benchmark recover closely aligned directions: absolute cosine similarities 0.81–0.95, against dictionary-wide reference medians 0.72–0.82. Cosine similarity compares the angle between two normalized directions. Across benchmarks the selected directions align at or below their reference medians, including when both dictionaries are compared at layer 18. Fresh dictionary seeds also reproduce positive r6.2 repair contrasts. This addresses dictionary randomness for fixed adapter representations; it does not establish reproducibility across training runs of the language model.

Does including malicious data in the dictionary create the finding? The original dictionaries include positive rows. Training new dictionaries only on benign rows reproduces the attribution pattern: at least 94 percent session mass on r4.2 and at least 99 percent profile mass in every r6.2 leave-one-user-out selection. The r4.2 ablation contrast replicates in all contexts, but its repair contrast becomes slightly negative. On r6.2, held-out repair and ablation depend on which user is tested. The selected locations are more reproducible than the individual intervention effects.

Is four versus sixty malicious users the explanation? Holding the r4.2 model and dictionary fixed, we reselected features from populations of 4, 10, 20, 30, and 60 malicious users, with ten repetitions per size. Profile share is about 0.25 at four users and 0.16–0.20 at larger sizes, far below r6.2. Count alone therefore does not explain selection in this fixed dictionary. The test does not isolate an effect of count on dictionary formation or prove a particular alternative explanation.

Does the pattern survive new adapters and a repaired serializer? Four file-routing fields were accidentally omitted from the original r6.2 serialization; r4.2’s release contains one of those fields. After repairing this defect, we trained new low-rank adapters on the pretrained Qwen2.5-3B model, using two seeds per benchmark and new benign-only dictionaries. The selected r4.2 features concentrate on sessions under both seeds. At the deepest tested r6.2 layer, three or four of the five features are almost entirely profile-bound, while shallower layers can be behavioral. The r4.2 ablation effect replicates under both seeds (+0.00218 and +0.00215); repair does not (about −0.0004 each). r6.2 intervention effects vary by seed and measurement. These are replications of the attribution pattern and r4.2 ablation dependence, not of every original causal estimate.

Is behavioral evidence absent from r6.2? No. Restricting selection to session positions and excluding profile-dominated features reveals a second feature family. It carries held-out repair on the dominant user (+0.0027–+0.0029), with partial held-out ablation dependence. Unrestricted selection prioritizes profile features; it does not show that the detector contains no behavioral causal structure.

Do input changes agree with the internal audit? Swapping a positive day’s profile with an unseen benign user’s profile changes mean NLL more than replacing all its session content on *both* benchmarks: +0.155 versus +0.072 on r6.2, and +0.180 versus +0.102 on r4.2. These probes measure sensitivity to an input change, not the benefits of familiar profiles, because donors were unseen. Thus raw score sensitivity is profile-dominated in both models even though their selected features activate at different locations. The masking and within-user tests help separate this sensitivity from useful discrimination.

7 Real data with real personality scores

The CERT data are simulated, including the personality scores. The natural objection is that the entire phenomenon might be an artifact of simulation. We therefore repeated the pipeline on TWOS, a public dataset collected at the Singapore University of Technology and Design from twenty-four real users over five days (Harilal et al., 2017), during which, by design of the underlying study, twelve users had their sessions temporarily taken over by other participants (a “masquerade” attack), and five users, after a staged dismissal, briefly acted as insiders on their own accounts (a “traitor” scenario). Each real user completed the standard fifty-item Big Five questionnaire, so the serialized inputs contain genuine psychometric information.

The details mirror the CERT setup at smaller scale. Mouse, keyboard, and logon telemetry are aggregated into ten-minute windows and serialized in the same three-line-class format, with the real Big Five scores forming the personality line (18,048 windows in total, of which 138 overlap a staged attack: 101 masquerade and 37 traitor windows). The model is the smaller Qwen2.5-3B, adapted on benign windows only (about seven thousand windows from the twelve users who were never masqueraded; for the five later dismissed users, only days before the dismissal enter training), trained for three epochs with two different adapter seeds; dictionaries are trained on benign rows only; selection and attribution proceed exactly as before.

This tests the attribution pattern in a population with real psychometrics and frequent staged attacks. Prevalence alone does not make identity uninformative; the within-user evaluation below supplies the direct control for a purely static identity score. The selected features place essentially all of their attribution mass on the behavioral lines, under both training seeds and at both audited layers (the smallest behavioral share of any selected feature is 0.983). Decisively, the evaluation compares each attacked user’s attack windows with that same user’s normal windows, holding static identity and personality information fixed. A score based solely on those static fields cannot explain the separation; the average within-user AUC is 0.78 across the sixteen attacked users (the twelve masqueraded users plus the four dismissed users with logged activity in their staged window). The intervention experiments reproduce the pattern seen in the benign-dictionary CERT replications: on the audited seed, the ablation contrast is +0.0019 and repair is −0.0018 over 138 receivers. Feature selection used the full pool, so these intervention results are exploratory. They support a real-data behavioral counterpart without certifying the entire detector as shortcut-free or demonstrating generalization to a new enterprise.

8 Real data with no profile at all

TWOS contains genuine profile measurements. A separate test asks whether a related failure appears in enterprise telemetry without a profile block. We use the “Comprehensive, Multi-Source Cyber-Security Events” dataset released by Los Alamos National Laboratory in 2015 (Kent, 2015): fifty-eight days of raw Windows authentication events from a real enterprise (12,425 users, 1.05 billion events), with 749 red-team compromise events (credential-based lateral movement) over 104 users as ground truth and no profile of any kind. Each event records only identity fields (source and destination user, source and destination host) and behavioral fields (authentication type, logon type, orientation, outcome, hour).

We serialized each user-day into windows of thirty-two consecutive events, one text line per event, and called a window positive if it contained a red-team event; the longest scored full-serialization window is 1,503 tokens against a 2,048-token context, and the longest scored positive is 1,417; these checked windows are below the limit. We kept every red-team user and a one-in-ten sample of benign users (1.62 million windows, 432 of them positive), adapted Qwen2.5-3B on benign windows only under the same recipe as the CERT experiments, and scored held-out windows by adapted negative log-likelihood. Users were hashed into five folds; the adapters trained on benign windows from four folds (a training budget of about 300,000 windows) and were evaluated separately on seen-fold users (22,563 windows, 234 positive) and on the held-out fold (30,410 windows, 198 positive). Four serializations differ only in the identity fields: the full line, user identity anonymized, host identity anonymized, and user identity deranged across users. Positive windows are rare (prevalence 1.0 percent among seen-fold windows and 0.65 percent among unseen-fold windows), so we report both AUC and average precision.

Input condition	Seen AUC	Seen AP	Unseen AUC	Unseen AP
full (user + host)	0.949	0.177	0.497	0.013
user anonymized	0.944	0.176	0.518	0.019
host anonymized	0.877	0.069	0.627	0.013
user deranged (shuffle)	0.947	0.173	0.488	0.012

The full, user-masked, and user-shuffled conditions have seen AUCs near 0.95 and unseen AUCs near 0.50. Host masking changes the trade-off: seen AUC falls to 0.877, while unseen AUC rises to 0.627. Unseen AP remains low (0.012–0.019 across conditions), roughly two to three times the 0.0065 positive fraction; host masking leaves AP nearly unchanged at 0.013. The ROC improvement therefore comes without a corresponding improvement in AP. No alert threshold or deployment precision is established by these aggregate metrics.

The host field is user-associated, not constant per user. In the audited training sample, the most frequent source computer accounts for a median 54 percent of a user’s events; only 7 percent of users generate at least 80 percent of events from one source computer. Hosts may encode identity proxies, legitimate resource use, and attack paths. The intervention identifies sensitivity to that field, without uniquely separating these explanations. A seen/unseen gap also remains after host masking (0.877 versus 0.627).

This extension broadens the evidence beyond synthetic profile fields, but it uses one adapter per condition and one held-out user fold. The ground truth is staged credential misuse, and users in the seen folds can contribute benign training windows. It is a complementary score-level experiment, not a replication of the CERT sparse-feature interventions. Appendix E records sampling and scoring details needed to interpret the comparison.

9 Which detectors take the shortcut?

One further question separates two very different readings of the r6.2 result. Is the profile information so predictive in that dataset that *any* statistical method would exploit it, or is the shortcut a property of this particular detector’s construction?

To answer it we trained three classical anomaly detectors on the same r6.2 days, represented as ordinary numeric feature vectors that *include* the profile variables (twelve profile columns among 242 input columns): an isolation forest, a one-class support vector machine, and a principal-component reconstruction method, each trained only on benign training users and evaluated on held-out users. The probe uses its own random user split rather than the language model’s hash split. For attribution we use permutation importance, which is defined without reference to any particular model class: for each input column, randomly permute that column’s values across the evaluation days, recompute the AUC, and record the decrease. Negative decreases are clipped to zero; a group’s importance is its share of the remaining total. These are descriptive one-permutation measurements.

All three classical detectors show little measured dependence on the profile variables in these tests. The profile columns receive four to seven percent of the total importance, no profile column appears among any detector’s five most important columns, and deleting the profile columns entirely does not reduce any detector’s AUC (it slightly increases it, by 0.003 to 0.023). The r4.2 result is mixed: one personality trait appears in the support vector machine’s top five, and removal lowers AUC for two of the three methods. Appendix F gives all six results. The r6.2 shortcut is therefore not an inevitable consequence of the available profile information: we do not observe comparable profile dominance in the three r6.2 classical baselines. One possible explanation is that repeated profile tokens are easy for a sequence model to predict during adaptation. However, this comparison changes the objective, representation, architecture, and size of the learner together. It shows that the observed reliance is not inevitable for these data; it does not isolate which of those differences causes it. Permutation importance and sparse-feature activation mass are also different measurements, so their percentages should not be equated.

10 What changes when the profile is removed before training?

The feature edits and input masks test a detector already trained with profiles. To test the effect of including those fields in the whole pipeline, we train a fresh adapter for each of four input conditions: FULL; NO-PSY, removing the personality line; NO-PROFILE, removing both DAY and PSY; and SHUFFLE, assigning each user another user’s profile through a fixed seeded derangement. A derangement is a permutation with no user keeping their own profile. It preserves a stable profile per user while breaking the original association with the recorded attributes.

We run all four conditions with Qwen2.5-3B and Qwen3-8B, using 300,000 benign examples and one adapter seed per condition and model. Each adapter is evaluated on the same unseen users. Each of the four malicious users is ranked against all 406 benign users in the eligible pool; user scores are maximum day scores and results average the four folds. The 800-user cap used earlier exceeds this pool. Each condition changes both adaptation and scoring, so the experiment measures their joint pipeline effect. It does not separate changes to learning from changes to the tokens included in the score.

Input condition	Qwen2.5-3B AUC		Qwen3-8B AUC	
	User	Day	User	Day
FULL	0.926	0.741	0.532	0.399
NO-PSY	0.903	0.718	0.906	0.714
NO-PROFILE	0.956	0.816	0.903	0.827
SHUFFLE	0.656	0.516	0.485	0.423

Removing profiles helps the audited 8B pipeline substantially. User AUC rises from 0.532 to 0.903: a paired change of +0.371 with a descriptive 95 percent interval of [+0.067, +0.690]. This bootstrap resamples only the four malicious users, holding the adapters and benign cohort fixed. Three users improve and one declines slightly; all four then rank in the top fifth of their 407-user comparison cohort. Removing just the personality line produces similar user AUC (0.906), but removing the entire profile improves day AUC further (0.827 versus 0.714). Thus the personality line accounts for much of the observed user-ranking loss; the result does not establish that the organizational header is irrelevant.

The 3B comparison limits a universal explanation. The 3B full-profile adapter already has user AUC 0.926. Profile removal changes it by +0.030 with interval [−0.065, +0.142]. Four of its five selected features are nevertheless profile-dominated. Representing profile information does not by itself establish a harmful shortcut. The contrast between the two model runs also does not isolate capacity: model generation and post-training differ along with size, and the experiment has one adapter seed per condition. The earlier 3B r6.2 reruns varied from 0.414 to 0.738 in day AUC across two seeds.

Shuffling distinguishes profile content from profile removal. Shuffling lowers user AUC relative to the original profile condition in both audited runs, while profile-dominated features remain among the top

five. The borrowed profile remains a stable pseudonym. A fixed one-to-one reassignment does not, by itself, cause a held-out user’s assigned profile to have appeared during training: ownership before the permutation is different from exposure after it. These results show that profile content matters to generalization in the tested pipelines, but do not identify whether stable identity, attribute–behavior associations, or particular profile similarities explain the shuffle loss.

A post hoc check of the four 8B users supplies a possible clue: the worst-ranked user has a personality vector close to many training users. However, the most novel profile also has only chance-level user ranking. The four cases support a familiarity hypothesis, without establishing a predictive rule. Appendix F.2 gives the per-user results, intervals, and novelty check so this qualification can be assessed directly.

11 Conclusions and limits

The central result is a separation between three questions: how well a detector ranks examples, which selected internal directions affect its score, and where those directions activate. The CERT audit finds a profile-concentrated feature set on r6.2 and a session-concentrated set on r4.2. Neither label fully describes the whole model. Both detectors retain within-user signal, and a secondary behavioral family exists on r6.2. Attribution is more stable across our replications than the repair effects of particular features.

The additional datasets and interventions suggest that population structure and the training representation influence the available shortcuts. They do not establish a two-factor law. Subsampling changes feature selection in an existing model; TWOS shows a behavioral counterpart with real questionnaires; LANL shows a host-sensitive score-level generalization gap; and the classical comparison changes several modeling choices together. The strongest direct input evidence is the improvement after removing profiles from adaptation and scoring in the audited 8B run. Its small comparison in 3B and the persistence of profile attribution under shuffling show why attribution alone is insufficient to diagnose harm.

For an audit, the practical sequence is simple: evaluate on users absent from training, inspect average precision and within-user ranking, test candidate features against active controls with same-user donors excluded, and check the proposed interpretation through input interventions. Report failed replications along with successful ones. Here the small r6.2 population, dictionary dependence, unequal edit sizes, and failed feature transfer materially limit what can be claimed.

Scope and reproducibility. The original 8B mechanism estimates concern one adapter per CERT release, one-layer edits, and the matched session domain. The headline dictionaries and configuration search use evaluation data; held-out feature selection is therefore conditional on them. The repaired 3B reruns and benign-only dictionaries provide additional checks, not independent replication of every 8B effect. TWOS is small and staged; LANL uses staged credential misuse and one held-out fold. No result establishes deployment readiness, a complete neural circuit, or a general effect of model size. The appendix records the protocols, quantitative controls, and artifact paths supporting these boundaries. Code and the reported result artifacts are available in the project repository; raw datasets and model checkpoints are separate inputs.

Implications for people. An insider-threat detector is also a workplace-monitoring system. Profile sensitivity can turn unusual personal attributes into suspicious scores, and an explanation can give weak evidence unwarranted authority. Benign-only training does not remove these risks. These experiments support auditing such systems, not autonomous action against individuals; any operational use would require appropriate human review and governance.

A Data, model, and dictionary details

A.1 What is included in the CERT evaluation?

The positive population is the intersection of the answer key and extracted session coverage: 70 user-days from 4 users on r6.2 and 1,309 from 60 on r4.2. The corresponding answer-key counts are 99 rows from 5 users and 1,883 from 70. The four r6.2 users contribute 46, 18, 5, and 1 positive days. Claims do not cover incidents outside the extracted domain.

The original language-model split excludes incident users from adaptation and assigns the remaining users by a deterministic hash. The validation comparison contains 405 benign users on r6.2 and 79 on r4.2. Other scored users can have no positive day in the matched domain. These differences matter: the later r6.2 factorial uses 406 negative users (405 validation users plus one evaluation user with no matched positive), and r4.2 scoring-time masking includes 89 never-trained benign users (79 validation users plus 10 others). The classical probe uses its own random user split (Appendix F).

A.2 Serialization and adapter training

The organizational line contains `week`, `project`, `role`, `b_unit`, `f_unit`, `dept`, `team`, `itadmin`; the personality line contains `O,C,E,A,N`. After `SESCOUNT`, session lines retain computer and timing descriptors, work-hour and weekend flags, duration, concurrency, logon and USB totals, USB duration, file and executable-file totals, email and attachment statistics, and HTTP categories including job, cloud, leak, and hacking sites. The original serializer omitted four hyphenated routing columns: `file_n-to_usb1`, `file_n-from_usb1`, `file_n-file_act3`, and `file_n-disk1`. The repaired r6.2 reruns restore all four; r4.2 contains one of them. The original experiments therefore concern the retained field set. Removing the entire `DAY` line also removes its `week` field, so that condition changes a temporal cue as well as organizational metadata.

All adapters use 4-bit NF4 base weights with double quantization and bfloat16 computation; LoRA rank 16, scale 32, dropout 0.05; attention $q/k/v/o$ and feed-forward up/down/gate projections; sequence limit 2048; cosine learning-rate decay with 3 percent warmup; and zero weight decay. The original Qwen3-8B recipe uses one epoch, effective batch 8, and learning rate 1.5×10^{-4} . Qwen2.5-3B uses effective batch 16 and learning rate 2×10^{-4} . The 8B factorial uses batch 16 across four GPUs while retaining its 1.5×10^{-4} rate. Recipes are held fixed across input conditions within each model; they are not identical across model families.

The repaired 3B runs use one epoch on all 287,827 r4.2 training days or a 300,000-day cap from approximately 1.25 million r6.2 days, with adapter seeds 42 and 43. TWOS uses three epochs on its smaller benign corpus. The factorial and LANL runs use a 300,000-example training cap and a 2,000-example validation cap, seed 42.

Scores average next-token negative log-likelihood over attention-valid targets; padding is excluded. When full fp32 logits would exceed 28 GiB, scoring computes cross-entropy in bounded slices from the hidden states and output embedding. This changes memory use, not the score’s mathematical definition. Input removal also changes which tokens enter that average, one reason the factorial is a pipeline-level effect.

A.3 Dictionary fitting and feature selection

Token deltas come from the residual stream after the audited layer, in evaluation-example order. For coordinatewise mean μ and standard deviation σ , the encoder and decoder are

$$z_t = \text{TopK}_k(\text{ReLU}(W_e((\delta_t - \mu)/\sigma) + b_e)), \quad \hat{\delta}_t = \sigma \odot Dz_t + \mu.$$

The decoder is untied and has no bias in standardized coordinates. Training uses Adam, learning rate 10^{-3} , 20 epochs, batches of 1024–4096, and at most two million token rows per layer. The original fit retains all positive-example rows and subsamples benign rows; benign-only dictionaries are separately identified.

Here m is the number of dictionary columns. The core search varies $m/d \in \{2, 4\}$, $k \in \{4, 8\}$, and layers 18, 26, 34 on r6.2 or 18, 26 on r4.2. The headline r6.2 choice is layer 18, $m/d = 4$, $k = 8$: standardized

reconstruction MSE 0.054, mean active count 7.98, and 1.0 percent dead features. The native r4.2 choice is layer 26, $m/d = 2$, $k = 4$: MSE 0.093, active count 4.0, and 1.4 percent dead features. These configurations were selected using exploratory intervention results on the full positive population.

The candidate set contains the five largest positive-minus-benign mean activation gaps. The active control contains five other features with smallest absolute gap among those active on at least 0.002 of token rows. Session-restricted selection recomputes gaps on **SES** rows and excludes features with more than 0.5 profile mass on positive rows. Held-out analyses reselect feature identities, but inherit the dictionary, layer, size, sparsity, context menu, and edit grid unless explicitly refitted.

B Detector measurements and input controls

B.1 Original baseline comparison

Each original fold (seed 42) holds out one malicious user and samples 800 benign users. The language model is fixed across folds and has trained on many of those benign users; baseline detectors retrain with test users excluded. Day metrics cover all test rows; user metrics use the maximum day score. Table 1 reports the fold averages. Deep SVDD is deep support vector data description (Ruff et al., 2018); GRU and LSTM AE are gated recurrent unit and long short-term memory autoencoders; IF is Isolation Forest (Liu et al., 2008). Their distinct training exposure prevents interpreting the LM’s higher ROC as a fair generalization advantage.

Dataset	Detector	Day ROC	Day AP	User AP	Mean rank
r6.2	Adapted LM	.953	.000755	.054	24.0
	Deep SVDD	.628	.01155	.211	83.2
	GRU AE	.766	.00572	.081	24.8
	LSTM AE	.768	.00207	.057	24.2
	IF	.713	.000211	.013	153.0
r4.2	Adapted LM	.964	.01345	.101	26.4
	Deep SVDD	.743	.03372	.382	53.4
	GRU AE	.696	.02544	.124	86.6
	LSTM AE	.714	.02364	.120	92.1
	IF	.715	.000254	.008	186.4

Table 1: Original fold-aligned comparison. Rank is the malicious user’s position among 801 users (smaller is better). AP is average precision, called PR-AUC in the result artifacts. Baseline user ROC is unavailable in this harness; adapted-LM fold averages are approximately .971 and .968 from the tracked rows. These are not the pooled values below.

Base-model NLL has day ROC 0.461 on r6.2 and 0.581 on r4.2 in the original comparison. Thus adaptation changes the ranking substantially, without establishing whether the acquired information is appropriate. In the full scored pool, restricting benigns to validation users changes day ROC from 0.954 to 0.545 on r6.2 and 0.959 to 0.668 on r4.2; user ROC changes from 0.970 to 0.546 and 0.969 to 0.565. The restricted day/user APs are 0.0005/0.013 and 0.1023/0.521. These APs have different class prevalence from the fold comparison and should not be treated as a direct cross-protocol performance gain.

B.2 Within-user ranking, masking, and swaps

Mean within-user ROC is 0.725 on r6.2, with user-bootstrap interval [0.654, 0.795] and all four users above chance; the dominant user has 0.768. On r4.2 the mean is 0.669, interval [0.619, 0.717], with 52 of 60 users above chance. These compare each user’s positive and own-benign days, not seen versus unseen identities.

Scoring-time masking uses a fixed adapter and a fixed unseen comparison. The r4.2 pool contains 42,468 days from 149 users, including the 89 benign users described above. Removing personality changes day ROC from 0.670 to 0.797 (paired change +0.127, interval [0.061, 0.193]), day AP from 0.069 to 0.112, and user

ROC from 0.666 to 0.775 (change +0.108, interval [0.038, 0.179]). Further masking organizational identifiers gives day change +0.063 with interval [-0.003, 0.130]. On r6.2, original/no-personality/ no-profile day ROC is 0.547/0.575/0.471; all paired change intervals contain zero. Masking intervals use 10,000 paired bootstrap draws, resampling users within both label strata. Scoring-time masking and the later removal-and-retraining experiment are distinct interventions.

The swap probe replaces either the positive day’s profile or its session content with an unseen validation user’s values, keeping the complementary part fixed. Mean signed NLL changes are +0.155 versus +0.072 on r6.2 and +0.180 versus +0.102 on r4.2. Both profile swaps increase mean NLL more. These replacements can disrupt natural profile–behavior combinations; they measure sensitivity, not an isolated familiarity mechanism or a causal feature’s semantic identity.

C Exact interventions and uncertainty

C.1 Repair and ablation

Let F be the candidate or control feature set and $T_F = \{t : \sum_{f \in F} z_{t,f} > 0\}$ the active receiver positions. For a donor d , the prototype p_f^d is the mean donor activation over positions active for F , falling back to all donor positions if that set is empty. For $f \in F$ and $t \in T_F$, repair sets

$$z'_{t,f} = (1 - \alpha)z_{t,f} + \alpha p_f^d, \quad \alpha \in \{0.25, 0.5, 0.75, 1\}.$$

Other coefficients are unchanged. At every token, the patched delta is $\delta'_t = \sigma \odot D z'_t + \mu$; a forward hook adds $\delta'_t - \delta_t$ to the adapted residual state. Thus both arms substitute a sparse reconstruction even away from edited positions. That shared baseline need not cancel after nonlinear downstream layers.

For receiver r and donor class $b \in \{\text{ben}, \text{anom}\}$, the recorded outcome is

$$\Delta_b^F(r) = \min_{d \in \mathcal{D}_b(r), \alpha} \{s(x_r; F, d, \alpha) - s(x_r)\}.$$

Negative values mean score reduction. For the same complete receivers $\mathcal{R}$, repair is

$$\tau = \frac{1}{|\mathcal{R}|} \sum_{r \in \mathcal{R}} [(\Delta_{\text{anom}}^S(r) - \Delta_{\text{ben}}^S(r)) - (\Delta_{\text{anom}}^C(r) - \Delta_{\text{ben}}^C(r))].$$

This is maximal achievable repair under the permitted search, not a donor-averaged causal effect. Donor pools are capped at 16 per receiver, drawn with a fixed random seed; malicious donors are scarcer (median 1 on r6.2, mean 13 on r4.2), creating unequal search opportunities.

Ablation replaces the edited coordinates by $(1 - \alpha)z_{t,f}$. For positive or matched-benign receiver q , let $e_F(q) = \min_{\alpha} \{s(x_q; F, \alpha) - s(x_q)\}$. Over complete positive/benign pairs (r, b) , the ablation contrast is

$$\nu = \overline{[e_S(b) - e_S(r)]} - \overline{[e_C(b) - e_C(r)]}.$$

Positive ν means more positive-specific score reduction from ablating the selected set than the control. It does not prove strict necessity of the features outside this operation.

Donor and receiver pairs match role, department×role, project×role, team, or department as specified by the context. All donors and benign receiver matches exclude the positive receiver’s user. Benign matches come from never-trained evaluation users. r4.2 repair has 1034–1157 complete receivers across its contexts. r6.2 team has no complete malicious-donor contrast and is unavailable. Every contrast uses complete paired outcomes, not separate arm means with different missingness.

C.2 What the control checks establish

The initial active controls are not matched in edit size. Typical best-row repair norms are approximately 71 versus 20 (selected versus control) on r6.2 and 112–124 versus 11–13 on r4.2. Within a feature set, benign-

and malicious-donor norms differ by only 0.2 percent on r6.2 and at most 6 percent on r4.2. These observations help diagnose the contrast but do not eliminate magnitude-related confounding. The reconstruction component has mean per-token norm approximately 7 and 37, respectively.

Controls additionally matched on activation frequency, magnitude, and decoder norm reduce the selected/control norm ratio to 1.33 on r6.2 and 1.79 on r4.2. The rerun results are:

	Repair τ [95% interval]	Ablation ν [95% interval]
r6.2	.0059 [−.0018, .0096]	.0609 [.0074, .0843]
r4.2	.00168 [.00105, .00228]	.0012 [−.0016, .0038]

The intervals are user-cluster bootstrap intervals; r6.2 has only four clusters. The r4.2 repair and r6.2 ablation contrasts retain intervals above zero. This is partial calibration, not exact equal-dose testing.

Replacing best-donor search with the median of each donor’s best-strength outcome yields positive headline repair contrasts of 0.0005 (r6.2) and 0.0006 (r4.2). Dividing each best-row score change by its edit norm gives r4.2 positive point estimates in three of four contexts (team +0.0008 per norm unit), but all context intervals include zero. r6.2 repair does not survive this rescaling (three usable user clusters). The rescaling assumes approximately linear response and is a sensitivity check, not an alternative identification proof.

Ablation sizes also differ: selected/control norms approximately 45/21 on r6.2 and 165–169/13–15 on r4.2. Selected features have fewer active tokens on benign matches (0.2–0.8 versus 3.0 on r6.2; about 2 versus 4.4 on r4.2). The receiver-class contrast is therefore partly a comparison between small and large edits. The matched-control and replication checks are material qualifications to its interpretation.

C.3 Uncertainty and selection

Primary intervention intervals use 10,000 draws of malicious users with replacement and recompute the pooled complete-case contrast. Users with more eligible days retain greater weight in the point estimate; clustering does not make it a mean of equally weighted users. Supplementary procedures use 4000 receiver/pair draws or bootstrap a mean of user means. The figures use the primary user-cluster result. All contexts are reported without a multiple-comparison correction; a strongest-context example is not confirmation of every context. Intervals condition on trained models and selected configurations, and four-user intervals are descriptive.

D Transfer and replication details

D.1 Literal transfer and geometric alignment

Transferring the r6.2 dictionary and feature sets directly to all 1309 r4.2 positive receivers gives negative repair contrasts in every audited configuration. The best values in the three transferred sparsity settings are −0.00094, −0.00105, and −0.00113. Native r4.2 rediscovery at layer 26 instead gives positive repair in all four contexts: team 0.00142, role 0.00111, department×role 0.00107, department 0.00098, with primary intervals above zero. Native ablation is positive with intervals above zero for department×role (0.0029) and role (0.0021), while team and department intervals cross zero (Figure 2). A local session autoencoder evaluated at the same user-day unit gives best repair contrasts 0.0011 on r6.2 and 0.00091 on r4.2. Its reconstruction-error score has different units from LM NLL, so these numbers do not quantify a relative causal advantage.

Feature alignment uses absolute cosine after undoing standardization: compare normalized $\sigma \odot D_{:,f}$. Each selected source direction is matched to the best target direction. The reference is the median best-match similarity across the whole source dictionary, not a formal statistical null test.

Comparison	Selected best matches	Dictionary reference
r6.2, three seed pairs	.807–.856	.720–.750
r4.2, three seed pairs	.837–.946	.814–.818
Across datasets, layers 18/26	.218–.236	.224–.405
Across datasets, layer 18	.295–.652	.370–.963

r4.2’s layer-18 comparison is away from its headline layer-26 mechanism. Fresh r6.2 dictionary seeds also yield positive repair in every finite context (role 0.0071 and 0.0019, versus original 0.0068). This tests dictionaries on the same cached representations.

D.2 Held-out users and benign-only dictionaries

r4.2 discovery/confirmation splits 60 malicious users into 30/30 (671/638 positive days). Four of five selected features agree with full-population selection. Held-out repair is 0.00034–0.00076; only department has a primary interval above zero (0.00076, [0.00035, 0.00121], 21/29 contributing users positive). Held-out ablation is directionally positive for role and department×role; team does not replicate a positive effect.

With the original r6.2 selection, project×role ablation is positive for all four users (0.019–0.103); pooled repair is largely driven by the 46-day user. In leave-one-user-out selection, the three folds retaining the dominant user select nearly identical sets; excluding that user changes the set completely. The new set repairs the dominant user’s held-out days (0.0037–0.0038), while effects for low-activity users are near zero. Held-out ablation is positive on the two other multi-day users (up to 0.043) but negative on the dominant-user fold. Intervals within a single held-out user describe variability over that user’s days, not independent-user replication.

With dictionaries fitted only to benign deltas, r4.2 attribution remains at least 94 percent session mass, with four features at 100 percent. Held-out ablation is +0.004–+0.013 across contexts (528–638 complete pairs), but repair is −0.0002–−0.0004. r6.2 remains at least 99 percent profile mass in every fold; repair is positive in one held-out fold, while ablation is positive on the dominant-user fold. Dictionary choice changes which intervention and user combination replicates.

D.3 Population, new adapters, and secondary features

For the fixed r4.2 model and dictionary, positive-user subsamples of 4, 10, 20, 30, 60 (ten draws each) give mean top-five profile share about 0.25 at four users and 0.16–0.20 otherwise. This is a selection-stage test conditional on a full-population dictionary.

The repaired 3B adapters use fresh deltas at layers 12, 18, 24 and benign-only dictionaries. On r4.2, all five features at the best layer have at least 94 percent session mass under both adapter seeds; unseen-user day ROC is 0.678/0.718. Ablation is +0.00218/0.00215 on 1301 complete pairs per seed, with repair about −0.0004 each. On r6.2, three or four of five layer-24 features are almost entirely profile-bound; shallower configurations can be behavioral. Day ROC is 0.414/0.738; repair is +0.0175 in one seed and near zero in the other. These runs jointly change model, serializer, and training realization.

r6.2 session-restricted leave-one-user-out selection recovers two recurring features across three of four folds, with near-zero profile mass. Held-out repair on the dominant user is +0.0027–+0.0029, around three-quarters of the unrestricted profile-family effect. Ablation replicates partially, including the dominant-user role context where the unrestricted profile-family result was negative. This shows a secondary session-concentrated family within the same model.

E The two real-data studies

TWOS contains 24 real users over five days with real IPIP-50 personality questionnaires. The serialization yields 18,048 ten-minute windows, 138 positive (101 masquerade and 37 traitor). The benign training pool is approximately 7000 windows from 12 users never masqueraded; future traitors contribute only pre-dismissal benign days. Both adapter seeds and audited layers 18/24 yield at least 0.983 session mass for each selected

feature. Within-user ROC averages 0.783 over the 16 attacked users with logged windows. The quoted interventions ($\nu = +0.0019$, $\tau = -0.0018$) use seed 42, layer 24, team matching, same-user exclusion, and 138 receivers. Dictionaries are benign-only but feature selection uses the full positive pool; the quoted point estimates are exploratory and are not accompanied here by independent-user confirmation.

LANL contains approximately 1.05 billion authentication events over 58 days. Keeping all 104 red-team users and a one-in-ten sample of other users yields approximately 1.62 million windows, 432 positive. A window contains up to 32 events within a user-day and is positive if it includes a red-team event. Five deterministic MD5 user folds assign fold 4 to unseen evaluation; benign windows from folds 0–3 supply training and validation. Evaluation keeps all positives and hash-samples benign windows, disjoint from the training and validation windows. Actual scored counts, rather than target sample budgets, are 22,563 seen-fold windows (234 positive) and 30,410 unseen-fold windows (198 positive). A seen-fold user can have benign training windows even if another window is red-team positive.

Token-length checks cover all 52,973 scored windows under the full and host-anonymized conditions: full maximum 1503, 99th percentile 1408, positive maximum 1417; host-anonymized maximum 1311, 99th percentile 1263, positive maximum 1212. All are below 2048. This check should not be expanded into a claim about every raw or generated window in all conditions. The host-constancy audit covers 9,414,753 training events and 1180 users with at least 20 events: modal source-host share has median 0.54, quartiles 0.40/0.61, mean 0.51; modal destination-host share has median 0.16. The full versus host-anonymized unseen AP is approximately 0.0133 versus 0.0129, at prevalence 0.00651. These values are above prevalence but show no AP improvement from host masking.

F Classical and train-matched comparisons

F.1 The separate classical probe

The classical probe aggregates numeric session columns to user-days: profile columns take the first value; behavioral columns take means, with session count added. It includes 12 profile and 230 behavioral columns on r6.2, 11 and 110 on r4.2. It randomly holds out 399/94 benign users, respectively (seed 42), and scores their days plus all matched positive days. This differs from the LM’s hash split. Training-day counts are 1,254,025/284,322 and evaluation-day counts 138,138/33,789.

Features are standardized on training data. Isolation Forest uses 300 trees; the one-class SVM uses $\nu = 0.05$, scaled kernel width, and a 6000-row training subsample; principal-component analysis (PCA) uses 10 components with reconstruction norm as score. The input-removal variants refit each model; SVM subsamples need not be identical between variants. Permutation importance is one random permutation per column, with negative ROC drops clipped to zero before computing group shares. These shares are descriptive and differ from sparse activation mass.

	Model	Profile share	Profile in top 5	Full ROC	Removal change
r6.2	IF	.040	0	.6408	+.0030
	One-class SVM	.065	0	.4914	+.0225
	PCA	.041	0	.6164	+.0220
r4.2	IF	.017	0	.8294	−.0084
	One-class SVM	.124	1	.6595	−.0223
	PCA	.024	0	.8382	+.0089

Profile reliance is small in the r6.2 probe. r4.2 is mixed: personality trait O enters the SVM’s top five, and removal lowers two detectors’ ROC. Thus the r6.2 pattern cannot be asserted unchanged on r4.2.

F.2 Factorial intervals and individual users

Each arm uses the same capped benign corpus and input condition during adaptation and scoring. The audit pool includes all evaluation-user days and a seeded 12 percent sample of validation days. Its fold re-score ranks each malicious user against the same 406 negative users. User AUC with one positive is a

measure of that user’s position relative to negatives, not the probability of detecting that user at a chosen alert threshold. Table 2 retains the individual results behind the body averages.

	Condition	ACM2278	CDE1846	CMP2946	MBG3183
3B	Full	.995 (3)	.803 (81)	.966 (15)	.941 (25)
	No personality	.916 (35)	.926 (31)	.924 (32)	.845 (64)
	No profile	.993 (4)	.993 (4)	.993 (4)	.845 (64)
	Shuffle	.894 (44)	.488 (209)	.520 (196)	.722 (114)
8B	Full	.919 (34)	.155 (344)	.527 (193)	.527 (193)
	No personality	.931 (29)	.872 (53)	.911 (37)	.911 (37)
	No profile	.867 (55)	.978 (10)	.951 (21)	.818 (75)
	Shuffle	.345 (267)	.241 (309)	.574 (174)	.778 (91)

Table 2: User ROC and rank in parentheses, out of 407 users per fold. All four users rank in the top fifth after profile removal in both models. This is a ranking result, not a demonstrated alert recall.

The 95 percent bootstrap intervals for user ROC are:

Condition	3B	8B
Full	[.844, .982]	[.248, .821]
No personality	[.865, .925]	[.882, .926]
No profile	[.882, .993]	[.842, .964]
Shuffle	[.504, .808]	[.293, .676]

Paired profile-removal changes are $+.030 [-.065, .142]$ and $+.371 [.067, .690]$. Resampling uses the same four malicious-user indices across contrasted arms and fixes the negative cohort. Bootstrap tail fractions in the raw logs are not additional training replications or a four-user significance guarantee.

In the separately fitted full/shuffle dictionaries, selected profile fractions are:

Model, layer, condition	Feature 1	2	3	4	5
3B, 12, full	1	.853	.997	.026	.992
3B, 12, shuffle	1	.920	1	1	.985
8B, 24, full	.053	.041	.095	1	1
8B, 24, shuffle	.085	.095	.257	.998	1

These are the capped factorial adapters, not the original layer-18 8B audit; each has its own selected features.

The exploratory novelty check compares each positive user with 3590 benign training users. In the table’s user order, counts of training profiles within Big-Five L_1 distance 10 are 3, 12, 11, 0; minimum distances are 9, 7, 6, 12; exact role/department/team matches are 29, 13, 22, 4. Corresponding full-8B day ROC values are 0.340, 0.191, 0.412, 0.653. The nearest-distance and neighborhood-count rankings differ, and neither perfectly orders user ROC. This four-case post hoc comparison cannot establish a general novelty mechanism.

G Relation to earlier work and artifact guide

Earlier insider-threat work models aggregated behavior and log sequences (Tuor et al., 2017; Le et al., 2020; Du et al., 2017; Guo et al., 2021). Event-level recurrent language models also use user context (Tuor et al., 2018); more recent systems include LoRA-based DMFI and LLM-based RedChronos (Kong et al., 2025; Li et al., 2025). Facade uses benign-trained contextual anomaly detection with a different objective and operational setting (Kantchelian et al., 2024). Our results do not establish that these systems share the audited failure. Context can be informative; the question is whether its use supports the intended generalization and alerting task.

Sparse dictionaries, activation interventions, and analyses of LoRA delta geometry have prior precedents (Gao et al., 2024; Zhang & Nanda, 2023; Prasanth, 2026). Our contribution combines these tools with user-

exposure controls and input-location attribution for a benign-adapted anomaly score. One-layer edits do not recover a complete circuit or fully test a computational hypothesis as in causal scrubbing (Chan et al., 2022). The contribution is therefore the combined empirical audit and its documented failure cases, not the individual tools or an unqualified claim to be the first use of delta sparse coding.

Where the evidence lives. Code and result artifacts are in the project repository. This edition uses revision `8a4e9d7`; `paper/companion/READABLE_COVERAGE.md` provides the section mapping and per-result source paths. For the factorial, the fold table and `anvil_run_summaries.txt` in `results/train_matched_factorial/` are the reported evidence; the nearby `roc/` files are earlier quick evaluations on a different population. LANL metrics and measurement checks are in `results/lanl_auth_replication/`. Raw datasets, pretrained weights, and trained models are separate dependencies. The tracked summaries support inspecting the contrasts; a fresh clone alone does not contain every input needed to rerun all GPU experiments.

References

- Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. Dos and don'ts of machine learning in computer security. In *31st USENIX Security Symposium (USENIX Security 22)*, pp. 3971–3988, 2022.
- Lawrence Chan, Adrià Garriga-Alonso, Nicholas Goldowsky-Dill, Ryan Greenblatt, Jenny Nitishinskaya, Ansh Radhakrishnan, Buck Shlegeris, and Nate Thomas. Causal scrubbing: A method for rigorously testing interpretability hypotheses, 2022. Alignment Forum.
- Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):15:1–15:58, 2009.
- Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023. arXiv preprint arXiv:2309.08600.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. In *Advances in Neural Information Processing Systems*, 2023.
- Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *ACM SIGSAC Conference on Computer and Communications Security*, 2017.
- Leo Gao, Tom Dupré de la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders, 2024. arXiv preprint arXiv:2406.04093.
- Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020.
- Joshua Glasser and Brian Lindauer. Bridging the gap: A pragmatic approach to generating insider threat data. In *IEEE Security and Privacy Workshops*, 2013.
- Haixuan Guo, Shuhan Yuan, and Xintao Wu. Logbert: Log anomaly detection via bert. In *International Joint Conference on Neural Networks*, 2021.
- Athul Harilal, Flavio Toffalini, John Castellanos, Juan Guarnizo, Ivan Homoliak, and Martín Ochoa. Twos: A dataset of malicious insider threat behavior based on a gamified competition. In *Proceedings of the 2017 International Workshop on Managing Insider Security Threats*, pp. 45–56, 2017.
- Stefan Heimersheim and Neel Nanda. How to use and interpret activation patching, 2024. arXiv preprint arXiv:2404.15255.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

-
- Alex Kantchelian, Casper Neo, Ryan Stevens, Hyungwon Kim, Zhaohao Fu, Sadegh Momeni, Birkett Huber, Cem Topcuoglu, Senaka Buthpitiya, Elie Bursztein, Yanis Pavlidis, Martin Cochran, and Massimiliano Poletto. Facade: High-precision insider threat detection using deep contextual anomaly detection. *arXiv preprint arXiv:2412.06700*, 2024.
- Alexander D. Kent. Comprehensive, multi-source cyber-security events. Los Alamos National Laboratory, <https://csr.lanl.gov/data/cyber1/>, 2015. doi:10.17021/1179829.
- Kaichuan Kong, Dongjie Liu, Xiaobo Jin, Guanggang Geng, Zhiying Li, and Jian Weng. DMFI: A dual-modality log analysis framework for insider threat detection with LoRA-tuned language models. *arXiv preprint arXiv:2508.05694*, 2025.
- Max Landauer, Florian Skopik, and Markus Wurzenberger. A critical review of common log data sets used for evaluation of sequence-based anomaly detection techniques. In *Proceedings of the ACM on Software Engineering (FSE)*, 2024. arXiv:2309.02854.
- Sebastian Lapuschkin, Stephan Wäldchen, Alexander Binder, Grégoire Montavon, Wojciech Samek, and Klaus-Robert Müller. Unmasking clever hans predictors and assessing what machines really learn. *Nature Communications*, 10(1):1096, 2019.
- Duc C. Le, Nur Zincir-Heywood, and Malcolm I. Heywood. Analyzing data granularity levels for insider threat detection using machine learning. *IEEE Transactions on Network and Service Management*, 17(1): 30–44, 2020.
- Chenyu Li, Zhengjia Zhu, Jiyan He, and Xiu Zhang. Redchronos: A large language model-based log analysis system for insider threat detection in enterprises. *arXiv preprint arXiv:2503.02702*, 2025.
- Brian Lindauer. Insider threat test dataset, 2020. Carnegie Mellon University, KiltHub repository. https://kiltHub.cmu.edu/articles/dataset/Insider_Threat_Test_Dataset/12841247.
- Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *IEEE International Conference on Data Mining*, 2008.
- K. Prasanth. Feature geometry of lora adapters: A sparse autoencoder analysis of representational divergence in fine-tuned language models. *arXiv preprint arXiv:2605.28896*, 2026.
- Qwen Team. Qwen3 technical report, 2025. arXiv preprint arXiv:2505.09388.
- Lukas Ruff, Robert A. Vandermeulen, Nico Görnitz, Lucas Deecke, Shoaib Ahmed Siddiqui, Alexander Binder, Emmanuel Müller, and Marius Kloft. Deep one-class classification. In *International Conference on Machine Learning*, 2018.
- Aaron Tuor, Samuel Kaplan, Brian Hutchinson, Nicole Nichols, and Sean Robinson. Deep learning for unsupervised insider threat detection in structured cybersecurity data streams. *AAAI Workshop on Artificial Intelligence for Cyber Security*, 2017.
- Aaron Tuor, Ryan Baerwolf, Nicolas Knowles, Brian Hutchinson, Nicole Nichols, and Rob Jasper. Recurrent neural network language models for open vocabulary event-level cyber anomaly detection. In *AAAI Workshop on Artificial Intelligence for Cyber Security*, 2018. arXiv:1712.00557.
- Fred Zhang and Neel Nanda. Towards best practices of activation patching in language models: Metrics and methods, 2023. arXiv preprint arXiv:2309.16042.